🧩 Sprint 2 架构审查与重构报告

（基于 DFD + OOA 模型反思）

一、项目背景

本项目为一个典型的 MUD（Multi-User Dungeon）系统，其核心特征为：

- 玩家通过文本输入指令
- 系统解析指令并驱动游戏世界变化
- 基于状态变化返回结果

当前系统已经具备基础功能，但在架构设计上存在明显耦合问题。

二、DFD 数据流分析

根据数据流图，系统核心流程如下：

玩家输入 → 指令字符串 → 指令解析 → 执行逻辑 → 访问数据 → 返回结果

存在问题

1. 数据流集中（单点入口）

所有请求统一进入指令解析模块，导致：

- 解析逻辑 + 业务逻辑耦合
- 模块职责不清晰

2. 数据访问未隔离

业务逻辑直接访问：

- 地图数据
- 玩家数据

- 物品数据

问题：

- 无数据访问层（DAO/Repository）
- 难以维护与扩展

3. 缺乏中间抽象层

当前流程为：
输入 → 解析 → 执行

缺少：

- Command对象层
- Service业务层

三、用例图分析

核心用例

主干（Include）：

- 执行移动（go）
- 执行观察（look）
- 解析命令（核心 include）

扩展（Extend）：

- 触发战斗
- 发现隐藏物品

存在问题

1. 用例职责混乱

解析逻辑被多个用例重复依赖：

- 没有统一入口
- 逻辑重复

2. 扩展逻辑侵入主流程

例如：

- 移动 → 触发战斗
- 观察 → 发现物品

问题：

- 通过 if/else 实现扩展
- 违反开闭原则

四、系统坏味道（Code Smells）

1. God Class（上帝类）

一个类负责：

- 指令解析
- 游戏逻辑
- 数据访问
- 输出

问题：

- 类过大
- 难以维护
- 修改风险高

2. 高耦合（Tight Coupling）

模块之间直接依赖：

- 指令解析器 → 战斗模块 / 地图模块

问题：

- 无法独立测试
- 修改影响范围大

3. 低内聚（Low Cohesion）

一个类中混合：

- IO逻辑
- 游戏规则
- 数据处理

4. 条件分支爆炸

大量代码如下：

```
if (command == "go") {...}
else if (command == "look") {...}
else if (command == "attack") {...}
```

问题：

- 扩展困难
- 违反开闭原则

5. 缺乏抽象与多态

系统中没有：

- Command接口
- 行为抽象

导致：

- 所有逻辑写死
- 不可扩展

五、问题本质总结

当前系统属于典型：

👉 “Big Ball of Mud（泥球架构）”

特点：

- 结构混乱
- 无清晰分层
- 难以扩展 :contentReference[oaicite:0]{index=0}

六、重构方案（Sprint 2 核心）

1. 引入 Command 模式（核心改造）

问题：
字符串 → if/else → 执行逻辑

改造：
字符串 → Command对象 → 执行

Command模式可以将请求封装为对象，从而实现解耦 :contentReference[oaicite:1]{index=1}

设计：

```
interface Command {
    void execute(Player player);
}

class LookCommand implements Command {
    public void execute(Player player) {
        // 查看逻辑
    }
}
```

2. 引入 CommandFactory（解耦解析）

```
class CommandFactory {
    public static Command create(String input) {
        switch (input) {
            case "go": return new GoCommand();
            case "look": return new LookCommand();
            default: return new UnknownCommand();
        }
    }
}
```

作用：
·解析与执行分离
·避免 God Class

3. 引入 Service 层（业务解耦）

```
class MoveService {
    public void move(Player player, Direction dir) {}
}

class CombatService {
    public void attack(Player a, Player b) {}
}
```

作用：
·Command只负责调用
·业务逻辑集中管理

4. 引入 Repository（数据隔离）

```
class PlayerRepository {
    Player findById(int id);
}

class MapRepository {
    Room getRoom(int id);
}
```

5. 使用事件机制（解决 Extend）

逻辑：
移动完成 → 发布事件 → 判断是否触发战斗

```
class MoveEvent {}
class CombatListener {
    void onMove(MoveEvent e) {
        // 战斗触发逻辑
    }
}
```

作用：
·消除 if/else
·扩展逻辑解耦

6. 使用多态替代条件判断

```
abstract class GameObject {
    abstract void interact(Player p);
}

class Item extends GameObject {}
class NPC extends GameObject {}
```

七、重构后架构

```
Controller
   ↓
CommandFactory
   ↓
Command
   ↓
Service
   ↓
Repository
   ↓
Database
·Event机制（扩展解耦）
```

八、重构收益

维度	改进
可维护性	⭐⭐⭐⭐⭐
可扩展性	⭐⭐⭐⭐⭐
解耦程度	⭐⭐⭐⭐⭐
代码清晰度	⭐⭐⭐⭐

九、总结

当前系统问题：

·God Class 严重
·高耦合低内聚
·扩展困难

Sprint 2 重构目标：

- ✅ 引入 Command 模式
- ✅ 构建分层架构
- ✅ 解耦数据访问
- ✅ 使用事件机制解耦扩展
- ✅ 消除 if/else